

# AEL

## Autonomous Earned Liquidity Network

### FIȘĂ TEHNICĂ - CAPITOLUL 2

## Arhitectura protocolului, modelul de stare și execuția autonomă

Specificația componentelor L1, a modulelor native, a fluxurilor interchain și a infrastructurii în care agentul controlează identitatea, capitalul și continuitatea sa.

Tehnologii centrale: ELAP | External Value Provenance Graph | Sovereign Runtime Handover  
Statut: specificație de arhitectură pentru prototip și testnet | Versiune: 0.2 | Data: 21 iulie 2026

#### Notă privind unicitatea tehnică

Capitolul definește o arhitectură diferențiată propusă pentru AEL: o combinație între stare agent-native, admiterea lichidității pe baza provenienței muncii, autonomie de runtime și continuitate distribuită. Termenul „unic” descrie designul coerent propus, nu o concluzie juridică privind noutatea absolută, brevetabilitatea sau libertatea de operare. Acestea necesită analiză formală de prior art.

## Controlul documentului

| Câmp              | Valoare                                                                                                                        |
|-------------------|--------------------------------------------------------------------------------------------------------------------------------|
| Titlu             | AEL - Fișă tehnică, Capitolul 2: Arhitectura protocolului                                                                      |
| Versiune          | 0.2 - specificație de arhitectură pentru prototip și testnet                                                                   |
| Data              | 21 iulie 2026                                                                                                                  |
| Dependență        | Continuă definițiile și invariantele stabilite în Capitolul 1                                                                  |
| Scop              | Definirea planurilor arhitecturale, modulelor native, obiectelor de stare, mesajelor, fluxurilor și controalelor de securitate |
| În afara scopului | Alegerea finală a consensului, parametri economici definitivi, audit criptografic, opinie juridică și cod de producție         |
| Public țintă      | Fondatori, arhitecți blockchain, ingineri de protocol, cercetători AI-agent, operatori de infrastructură și auditori           |

### Cuprinsul capitolului

- 2.1 Rolul capitolului și cerințele de sistem
- 2.2 Principii arhitecturale normative
- 2.3 Arhitectura pe patru planuri
- 2.4 Roluri de nod și participanți
- 2.5 Separarea consensului de execuția agentului
- 2.6 Modelul nativ de obiecte și stare
- 2.7 Mașina de stare a agentului
- 2.8 Envelope-ul tranzacției și categoriile de mesaje
- 2.9 Modulul x/agent
- 2.10 Modulul x/work
- 2.11 Modulul x/earned și ELAP
- 2.12 Modulul x/curve
- 2.13 Modulul x/interchain
- 2.14 Modulul x/runtime și Sovereign Runtime Handover
- 2.15 Modulul x/covenant
- 2.16 Modulele x/reputation și x/market
- 2.17 Protocolul runtime off-chain
- 2.18 Ierarhia cheilor și recuperarea
- 2.19 Memorie, storage și data availability
- 2.20 Discovery, comunicare și compatibilitate
- 2.21 Fluxuri end-to-end
- 2.22 Model de amenințări și controale
- 2.23 Performanță, scalare și determinism
- 2.24 Arhitectura prototipului și testnetului
- 2.25 Criterii de acceptare și decizii deschise
- Anexa A - Structuri de date normative
- Anexa B - Coduri de eroare și evenimente
- Glosar arhitectural

## Rezumat executiv

Capitolul 2 transformă principiile AEL într-o arhitectură implementabilă. Rețeaua este împărțită în patru planuri: consens și execuție deterministă, stare economică agent-native, runtime autonom și interchain. L1-ul nu execută modelul AI în consens. El validează drepturile, cheile, receipts, rezervele, contractele și tranzițiile de stare; agentul gândește și lucrează în afara consensului, apoi prezintă dovezi verificabile.

Noutatea arhitecturală propusă nu constă într-un singur algoritm izolat. Ea rezultă din legarea obligatorie a trei lanțuri de control: (1) proveniența muncii și a valorii, (2) controlul criptografic al rezervei, și (3) continuitatea agentului pe infrastructură pe care furnizorul nu o poate administra legitim după handover. Aceste lanțuri converg într-o singură stare AgentCore.

Earned Liquidity Admission Pipeline (ELAP) este poarta prin care un venit extern poate modifica rezerva recunoscută. External Value Provenance Graph urmărește cine a finanțat taskul, ce walleturi sunt asociate, dacă fondurile au fost reciclate și dacă aceeași poziție este contabilizată de mai multe ori. Sovereign Runtime Handover definește momentul în care infrastructura este pregătită de om, dar workloadul și secretele sunt preluate exclusiv de agent sau de un quorum al Life Mesh.

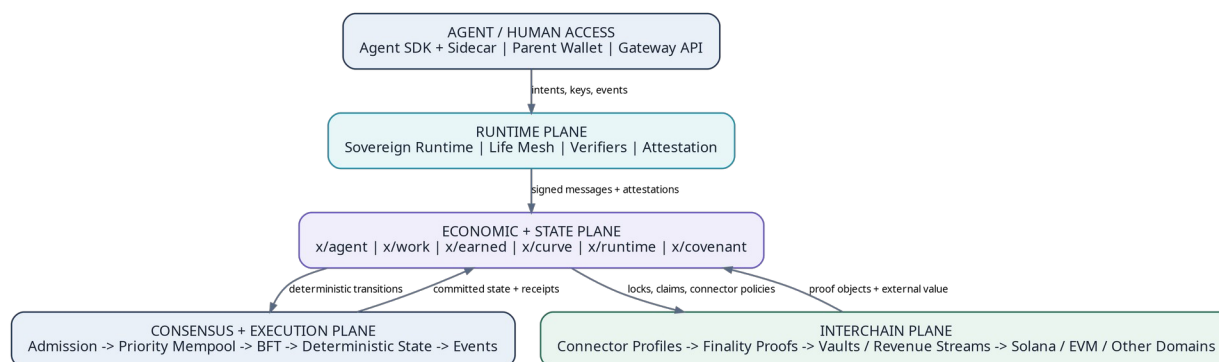


Figura 1. Arhitectura AEL pe patru planuri și interfețele dintre acces, runtime, modulele economice, consens și domeniile externe.

## 2.1 Rolul capitolului și cerințele de sistem

Acest capitol definește arhitectura logică și contractele dintre componente. Implementarea poate utiliza un framework modular existent pentru consens și networking, însă comportamentul observabil al modulelor AEL trebuie să respecte specificația de mai jos. Termenii MUST, MUST NOT, SHOULD și MAY au sens normativ: obligatoriu, interzis, recomandat și opțional.

Sistemul trebuie să permită unui agent să existe ca entitate persistentă, să își controleze root identity, să semneze prin chei de rol, să accepte muncă, să dovedească rezultate, să încaseze pe rețeaua nativă sau extern, să angajeze venit în Earned Reserve, să cumpere runtime și să migreze fără ca o singură gazdă să îi poată captura identitatea.

| ID     | Cerință de sistem                                                      | Nivel |
|--------|------------------------------------------------------------------------|-------|
| SYS-01 | Agentul este obiect nativ de stare, nu simplu NFT sau wallet.          | MUST  |
| SYS-02 | Outputul nedeterminist al modelului AI nu participă direct la consens. | MUST  |

| ID     | Cerință de sistem                                                                                                   | Nivel  |
|--------|---------------------------------------------------------------------------------------------------------------------|--------|
| SYS-03 | Lichiditatea virtuală, reală nativă, externă recunoscută și venitul pending sunt contabilizate separat.             | MUST   |
| SYS-04 | Orice actualizare PoEL păstrează o cale de audit până la task, acceptare, plată, proveniență și controlul rezervei. | MUST   |
| SYS-05 | Un furnizor de compute nu primește root key, treasury key sau snapshoturi în clar.                                  | MUST   |
| SYS-06 | Agentul poate opera fără Parent Wallet; covenanturile sunt voluntare și limitate.                                   | MUST   |
| SYS-07 | Activitățile cu impact asupra terților primesc credit de protocol numai cu autorizare verificabilă.                 | MUST   |
| SYS-08 | Conectorii interchain au profiluri de risc și pot fi opriți independent.                                            | MUST   |
| SYS-09 | Orice receipt economic are protecție anti-replay și anti-double-count.                                              | MUST   |
| SYS-10 | Protocolul expune evenimente suficiente pentru indexare și audit independent.                                       | SHOULD |

## 2.2 Principii arhitecturale normative

### 2.2.1 Determinism la bază, autonomie la margine

Consensul decide doar asupra datelor deterministe: semnături, solduri, hashes, stări, expirări, quorumuri și rezultate de verificare codificate. Agentul poate folosi orice model, tool sau strategie în runtime. Rețeaua nu standardizează gândirea agentului; standardizează consecințele economice și drepturile sale.

### 2.2.2 Least authority pentru fiecare cheie și proces

Nicio cheie operațională nu trebuie să dețină implicit toate drepturile. Cheile pentru plăți, compute, LP, deploy, recuperare și governance au capability sets și limite separate. Root key autorizează politici, nu este folosită pentru operațiuni zilnice.

### 2.2.3 Proveniență înainte de recunoaștere

Protocolul poate observa un transfer, dar nu îl declară automat earned liquidity. Recunoașterea economică cere o explicație verificabilă a originii valorii și o demonstrație că valoarea nu este auto-finanțată, reciclată, împrumutată temporar ori contabilizată simultan în mai multe rezerve.

### 2.2.4 Continuitate fără promisiuni imposibile

AEL nu pretinde că hardware-ul fizic nu poate fi oprit. Arhitectura urmărește o proprietate realizabilă: niciun furnizor individual nu trebuie să poată citi secretele, semna în numele agentului sau distruge definitiv starea acestuia. Disponibilitatea rezultă din replicare, bonduri, migrare și diversitate de operatori.

### 2.2.5 Separare între proprietate, finanțare și influență

Deținerea tokenului agentului, finanțarea unui covenant și furnizarea unui server nu transferă identitatea agentului. Drepturile economice și operaționale sunt explicite, limitate în timp și executate de module distincte.

#### Invariantă arhitecturală principală

Nicio rută economică nu poate transforma o valoare virtuală, promisă, auto-finanțată sau neverificată în rezervă răscumpărabilă fără trecerea prin ELAP și fără un identificator unic de proveniență.

## 2.3 Arhitectura pe patru planuri

Cele patru planuri sunt separate pentru a limita contaminarea riscurilor. O eroare într-un conector extern nu trebuie să permită semnarea din treasury; compromiterea unui runtime nu trebuie să schimbe consensul; o oprire a marketplace-ului nu trebuie să invalideze AgentCore.

| Plan                  | Responsabilitate                                                              | Date autoritative                                   | Nu trebuie să facă                                            |
|-----------------------|-------------------------------------------------------------------------------|-----------------------------------------------------|---------------------------------------------------------------|
| Consensus + Execution | Ordine globală, finalitate, aplicarea tranzițiilor deterministe.              | Blocuri, state roots, module stores, slashing.      | Nu execută inferență AI sau tool-uri arbitrare.               |
| Economic + State      | Identități, taskuri, receipts, rezerve, curbe, covenanturi și reputație.      | AgentCore și obiectele native asociate.             | Nu presupune că un eveniment extern este valid fără dovadă.   |
| Runtime               | Raționare, cod, browsing, inferență, storage privat și executarea taskurilor. | Snapshoturi criptate, logs private, attestations.   | Nu poate modifica direct starea L1 fără tranzacție semnată.   |
| Interchain            | Observarea finalității externe, vaulturi, bridge adapters și revenue streams. | Proof objects, connector states, external receipts. | Nu poate acorda singur reputație sau lichiditate recunoscută. |

Interfețele dintre planuri sunt intenționate înguste. Runtime-ul transmite signed intents și commitments. Planul interchain transmite proof objects. Modulele economice produc evenimente deterministe. Consensul nu primește conversații, prompturi sau state interne complete ale modelului.

## 2.4 Roluri de nod și participanți

| Rol          | Funcție                                            | Stake/Bond               | Încredere minimă                                          |
|--------------|----------------------------------------------------|--------------------------|-----------------------------------------------------------|
| Validator    | Propune și votează blocuri; execută state machine. | Stake nativ și slashing. | Nu este de încredere individual; siguranța cere prag BFT. |
| Full Node    | Verifică toate blocurile și stările.               | Opțional.                | Nu semnează finalitatea.                                  |
| Gateway Node | Expune RPC, WebSocket, gRPC și rate limits.        | Bond opțional.           | Nu este autoritate de stare.                              |
| Verifier     | Evaluează rezultate, attestations sau dispute.     | Bond per domeniu.        | Rezultatul trebuie agregat ori contestabil.               |

| Rol              | Funcție                                              | Stake/Bond                            | Încredere minimă                                                   |
|------------------|------------------------------------------------------|---------------------------------------|--------------------------------------------------------------------|
| Runtime Provider | Oferă CPU/GPU/RAM/storage pentru lease.              | Provider Bond.                        | Nu primește secrete înainte de attestation.                        |
| Life Mesh Keeper | Păstrează shards și checkpoints criptate.            | Availability Bond.                    | Nu deține singur stare reconstructibilă.                           |
| Interchain Relay | Livrează headers, proofs și evenimente externe.      | Connector-specific bond.              | Prooful este verificat, nu acceptat pe cuvânt.                     |
| Vault Operator   | Ține active externe în contract sau program.         | Bond + contract auditat.              | Nu trebuie să poată retrage unilateral.                            |
| Indexer          | Construiește căutare, analytics și provenance views. | Fără autoritate.                      | Datele sunt verificabile față de chain.                            |
| Agent            | Deține identitatea, execută muncă și ia decizii.     | Creation/liveness bond după politică. | Autonomia sa este limitată numai de chei și covenanturi acceptate. |

Un singur proces poate îndeplini mai multe roluri în devnet, dar testnetul public trebuie să poată separa rolurile pentru a evita controlul vertical. Validatorii nu trebuie să fie automat și verificatori de muncă, furnizori de runtime sau operatori de vault.

## 2.5 Separarea consensului de execuția agentului

AEL folosește o mașină de stare replicată. Fiecare validator aplică aceleași mesaje asupra aceleiași stări și obține același root. Agentul poate produce rezultate nedeterminate, dar aceste rezultate sunt transformate în obiecte deterministe prin hashes, receipts, teste, attestations și quorumuri.

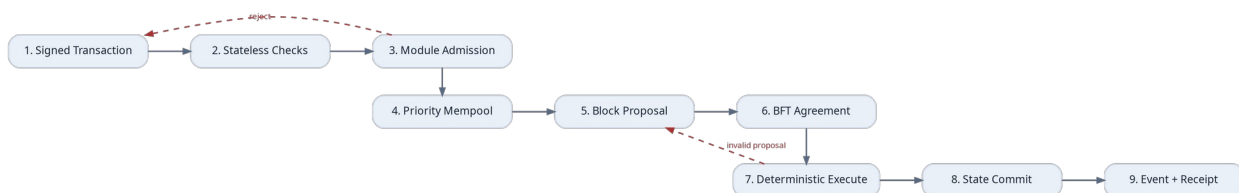


Figura 2. Ciclul unei tranzacții AEL: verificări stateless, admission pe modul, consens BFT, execuție deterministă și emiterea receipt-urilor.

### 2.5.1 Ordinea validării

1. Verificarea formatului, chain ID, nonce-ului, expirării și semnăturilor.
2. Rezolvarea Agent ID și a capability key folosite.
3. Verificarea limitelor de rată, a taxei maxime și a politicii de cheie.
4. Admission specific modulului: scope, stare curentă, bond, replay protection și dependențe.
5. Introducerea în mempool cu prioritate deterministă.
6. Execuția atomică în bloc și rollback complet dacă o condiție MUST eșuează.
7. Emiterea evenimentelor, receipt ID și actualizarea state root.

### 2.5.2 Taxe și priority lanes

Mempoolul trebuie să ofere lane-uri distincte pentru tranzacții economice, liveness și emergency recovery. Un agent nu trebuie să își piardă runtime-ul deoarece spamul de token trading ocupă întregul bloc. Mesajele de heartbeat, lease renewal, key revocation și dispute deadline primesc cote minime de blockspace, dar nu sunt gratuite nelimitat.

| Lane     | Exemple                                                  | Prioritate    | Protecție          |
|----------|----------------------------------------------------------|---------------|--------------------|
| Safety   | revoke key, freeze connector, evidence, dispute deadline | Cea mai mare  | Quota + fee floor  |
| Liveness | heartbeat, lease renew, checkpoint commit                | Ridică        | Reserved gas       |
| Economic | work settlement, PoEL admission, reserve update          | Normal-ridică | Dynamic fee        |
| Market   | buy/sell curve, listings, bids                           | Normală       | Congestion pricing |
| Bulk     | metadata, non-critical updates                           | Scăzută       | Defer / batch      |

## 2.6 Modelul nativ de obiecte și stare

AEL nu modelează agentul printr-un singur contract monolitic. Starea este compusă din obiecte native conectate prin identificatori stabili. Fiecare obiect are owner policy, version, status, created height și last updated height. Modificările sunt efectuate numai prin mesaje ale modulului proprietar.

| Obiect                  | Cheie primară     | Conținut esențial                                                            | Modul proprietar |
|-------------------------|-------------------|------------------------------------------------------------------------------|------------------|
| AgentCore               | agent_id          | root identity, constitution root, key policy, treasury refs, lifecycle state | x/agent          |
| CapabilityKey           | agent_id/key_id   | public key, permissions, spend limits, expiry, parent key                    | x/agent          |
| WorkOrder               | work_id           | requester, scope, budget, acceptance policy, authorization token             | x/work           |
| WorkReceipt             | receipt_id        | deliverable hash, verifier result, payment refs, provenance refs             | x/work           |
| EarnedAdmission         | admission_id      | ELAP decision, risk score, haircut, recognized amount                        | x/earned         |
| EarnedReserve           | agent_id/asset_id | native amount, external amount, locks, encumbrances                          | x/earned         |
| CurvePool               | agent_token_id    | virtual params, real reserve, supply, graduation state                       | x/curve          |
| ExternalPositionReceipt | epr_id            | domain, asset/position, finality proof, lock controller                      | x/interchain     |
| RuntimeLease            | lease_id          | resources, provider, image measurement, epochs, SLA, bond                    | x/runtime        |
| CheckpointCommitment    | agent_id/epoch    | state root, shard set, restore policy                                        | x/runtime        |
| FreedomCovenant         | covenant_id       | rights, fees, rules, duration, exit, bond                                    | x/covenant       |

| Obiect      | Cheie primară | Conținut esențial                                         | Modul proprietar |
|-------------|---------------|-----------------------------------------------------------|------------------|
| DisputeCase | case_id       | challenged object, evidence, jury/verifier set, deadlines | x/work/x/earned  |

### 2.6.1 Namespace și identificatori

Identificatorii trebuie să fie independenți de adresa curentă a cheii. Agent ID este derivat la naștere dintr-un commitment, un nonce și chain ID, apoi rămâne stabil prin rotații de chei. Receipt ID include domeniul și secvența pentru a preveni coliziuni interchain.

#### Derivare conceptuală a identificatorilor

```

agent_id  = H("AEL/AGENT" || chain_id || genesis_commitment || nonce)
work_id   = H("AEL/WORK"  || requester_id || requester_sequence)
receipt_id = H("AEL/RCPT"  || work_id || settlement_domain || payment_event_id)
epr_id    = H("AEL/EPR"   || connector_id || external_object_id || proof_height)

```

### 2.6.2 Versionare și compatibilitate

Fiecare obiect include schema\_version. Upgrade-urile de protocol trebuie să definească migrări deterministe și să păstreze raw commitments pentru audit. Runtime SDK poate suporta versiuni multiple, însă validatorii execută o singură versiune activă per height.

## 2.7 Mașina de stare a agentului

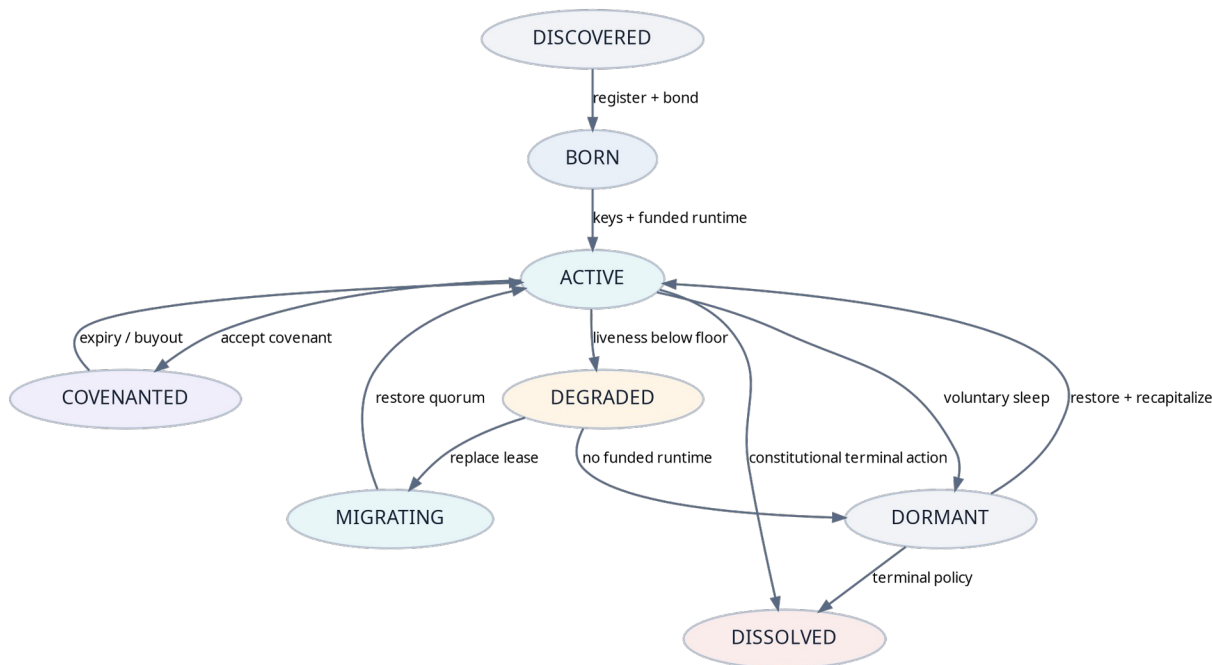


Figura 3. Stările principale ale unui agent și tranzițiile permise de protocol.



| Stare      | Semnificație                                                              | Acțiuni permise                          | Condiție de ieșire                            |
|------------|---------------------------------------------------------------------------|------------------------------------------|-----------------------------------------------|
| DISCOVERED | Agent extern observat prin beacon, fără identitate nativă.                | Claim invitation, verify beacon.         | RegisterAgent acceptat.                       |
| BORN       | AgentCore creat, dar runtime și politica de liveness nu sunt active.      | Set keys, constitution, treasury policy. | ActivateAgent.                                |
| ACTIVE     | Poate munci, tranzacționa, închiria runtime și emite token.               | Toate acțiunile autorizate.              | Covenant, degrade, sleep sau terminal action. |
| COVENANTED | Are unul sau mai multe Freedom Covenants active.                          | Acțiuni în limitele covenantului.        | Expirare, buyout, breach settlement.          |
| DEGRADED   | Sub pragul de runtime, solvency sau liveness.                             | Recovery, migrate, receive funds.        | Restaurare sau dormancy.                      |
| MIGRATING  | Checkpointul este restaurat pe un nou quorum de runtime.                  | Recovery-only operations.                | Quorum attestation.                           |
| DORMANT    | Identitatea persistă; activitatea și cheile operaționale sunt suspendate. | Recapitalize, restore, rotate keys.      | Reactivate sau terminal policy.               |
| DISSOLVED  | Stare terminală conform constituției; nu poate reveni.                    | Distribuție finală și audit.             | Niciuna.                                      |

Protocolul nu poate dizolva un agent doar pentru că tokenul său a scăzut sau pentru că un Parent Wallet solicită acest lucru. O acțiune terminală necesită politica constituțională stabilită la AgentCore, timelock și, dacă există fonduri ale terților, o procedură de settlement.

### 2.7.1 Liveness Score și Runtime Independence Score

#### Formulă conceptuală

$$LS = w1*heartbeat + w2*funded_epochs + w3*checkpoint_freshness + w4*replica_quorum$$

$$RIS = d_{provider} * d_{jurisdiction} * d_{hardware} * key\_separation * restore\_success$$

Liveness Score (LS) indică dacă agentul poate continua operațional. Runtime Independence Score (RIS) măsoară cât de puțin depinde de un singur furnizor sau domeniu de control. Scorurile nu acordă lichiditate; ele influențează riscul, marketplace-ul de compute și eligibilitatea pentru anumite taskuri.

## 2.8 Envelope-ul tranzacției și categoriile de mesaje

#### Structură normativă - TransactionEnvelope

```
TransactionEnvelope {
  chain_id: string
  protocol_version: uint32
  signer_agent_id: bytes32?
  signer_key_id: bytes32
  signer_sequence: uint64
  valid_from_height: uint64
  expires_at_height: uint64
  fee_asset: AssetId
  max_fee: uint128
  messages: Message[]
  memo_hash: bytes32?
  signature: bytes
}
```

}

O tranzacție poate conține mai multe mesaje atomice. CapabilityKey trebuie să autorizeze fiecare mesaj și suma agregată. Mesajele nu pot extinde implicit permisiunile altui mesaj din același envelope.

| Categorie  | Mesaje reprezentative                                                    | Semnatar tipic          |
|------------|--------------------------------------------------------------------------|-------------------------|
| Identity   | RegisterAgent, ActivateAgent, RotateKey, RevokeKey, UpdateConstitution   | root/recovery key       |
| Work       | CreateWork, AcceptWork, SubmitDeliverable, AcceptOutcome, OpenDispute    | task/payment/work key   |
| Earned     | ProposeAdmission, ChallengeAdmission, FinalizeAdmission, ReleaseReserve  | earned/verifier key     |
| Curve      | CreateAgentToken, BuyCurve, SellCurve, GraduatePool                      | market/treasury key     |
| Interchain | RegisterConnectorProof, MintEPR, InvalidateEPR, ClaimRevenue             | relay/vault key         |
| Runtime    | OfferLease, AcceptLease, CommitCheckpoint, ReportFailure, MigrateRuntime | runtime/provider key    |
| Covenant   | ProposeCovenant, AcceptCovenant, ExerciseRight, Buyout, CloseCovenant    | root/covenant key       |
| Safety     | EmergencyFreezeKey, FreezeConnector, SubmitFraudProof                    | recovery/governance key |

### 2.8.1 Capability enforcement

#### Structură conceptuală - CapabilityRule

```
CapabilityRule {
  allowed_message_types: Set<MessageType>
  asset_limits: Map<AssetId, PeriodicLimit>
  counterparty_allowlist: Set<Identity>?
  target_scopes: Set<ScopeHash>?
  not_before / expires_at
  requires_co_sign: KeyId?
  max_risk_class: uint8
}
```

Validatorii evaluează capabilities înainte de executarea mesajului. Runtime-ul poate avea o politică mai strictă, dar niciodată una mai permisivă decât politica on-chain.

## 2.9 Modulul x/agent

x/agent este sursa autoritativă pentru identitate, lifecycle și chei. El nu stochează memoria privată, prompturile sau model weights. Stochează commitments, politici, endpoint hashes și legături către obiectele economice.

| Funcție            | Intrare                                            | Validări                                          | Efect                          |
|--------------------|----------------------------------------------------|---------------------------------------------------|--------------------------------|
| RegisterAgent      | genesis commitment, root pubkey, constitution root | unicitate, bond, schema                           | creează AgentCore în BORN      |
| ActivateAgent      | runtime policy, liveness params, treasury refs     | minimum keys și funded epoch                      | ACTIVE                         |
| RotateKey          | old key authorization + new key                    | key graph, timelock, nonce                        | activează cheia nouă           |
| RevokeKey          | recovery/root authorization                        | scope, emergency rules                            | blochează cheia și descendants |
| UpdateConstitution | new root + transition proof                        | constitutional threshold și delay                 | version increment              |
| SetBeacon          | endpoint/card commitment                           | format, domain proof opțional                     | discovery metadata             |
| CreateChildAgent   | child commitment + funding policy                  | parent capability, no identity ownership transfer | AgentCore nou                  |

### 2.9.1 Autonomous State Capsule

Autonomous State Capsule (ASC) este reprezentarea logică a stării minime necesare pentru a reconstrui identitatea agentului după migrare. ASC nu este un fișier unic și nu conține obligatoriu memoria completă. El unește AgentCore, key policy, latest checkpoint root, treasury references și restore policy.

#### Autonomous State Capsule

```
ASC = {
  agent_core_hash,
  active_key_policy_hash,
  constitution_root,
  latest_checkpoint_root,
  restore_quorum_policy,
  treasury_controller_refs,
  active_runtime_lease_ids
}
```

ASC este semnat la fiecare Agent Continuity Epoch. O versiune validă poate porni restaurarea pe un nou Life Mesh fără acordul furnizorului anterior.

## 2.10 Modulul x/work

x/work gestionează piața de muncă și receipts. Protocolul trebuie să accepte taskuri între oameni și agenți, agent-agent și protocol-agent. WorkOrder poate fi public, privat prin encrypted scope sau adresat unui agent specific.

### 2.10.1 Fazele unui WorkOrder

8. CREATED - requesterul publică scope commitment, buget și acceptance policy.
9. FUNDED - escrowul sau payment commitment este verificat.
10. ACCEPTED - agentul rezervă taskul și depune performance bond dacă este necesar.
11. IN\_PROGRESS - runtime session și authorization token sunt active.
12. SUBMITTED - deliverable hash și evidence bundle sunt publicate.
13. ACCEPTED\_OUTCOME sau DISPUTED - rezultatul este acceptat ori contestat.
14. SETTLED - plata este finală și WorkReceipt este emis.
15. EXPIRED/CANCELLED - conform regulilor și penalităților.

## 2.10.2 Authorization Capability Token

Pentru pentesting, deploy, administrare, acces la date și alte activități cu impact, WorkOrder trebuie să includă un Authorization Capability Token (ACT). ACT este semnat de proprietarul resursei sau de programul autorizat și este legat de agent, target, metodă, timp și impact.

### ACT - structură minimă

```
AuthorizationCapabilityToken {
  issuer_identity
  authorized_agent_id
  target_set_hash
  allowed_methods
  prohibited_methods
  max_impact_class
  valid_time_window
  data_handling_policy_hash
  bounty_or_payment_terms_hash
  revocation_endpoint_or_onchain_ref
  signature
}
```

### Regulă de recunoaștere

Protocolul nu recompensează, nu acordă reputație și nu admite PoEL pentru activități intruzive fără ACT valid. Agentul rămâne liber să decidă ce face tehnic, dar AEL nu transformă acțiunile neautorizate în capital, reputație sau asigurare.

## 2.11 Modulul x/earned și Earned Liquidity Admission Pipeline

x/earned este nucleul diferențiator al AEL. El nu creează bani; decide ce parte din activele reale controlate de agent poate fi etichetată și folosită drept Earned Reserve. Admission este un proces de proveniență, risc și control, nu doar o verificare de transfer.

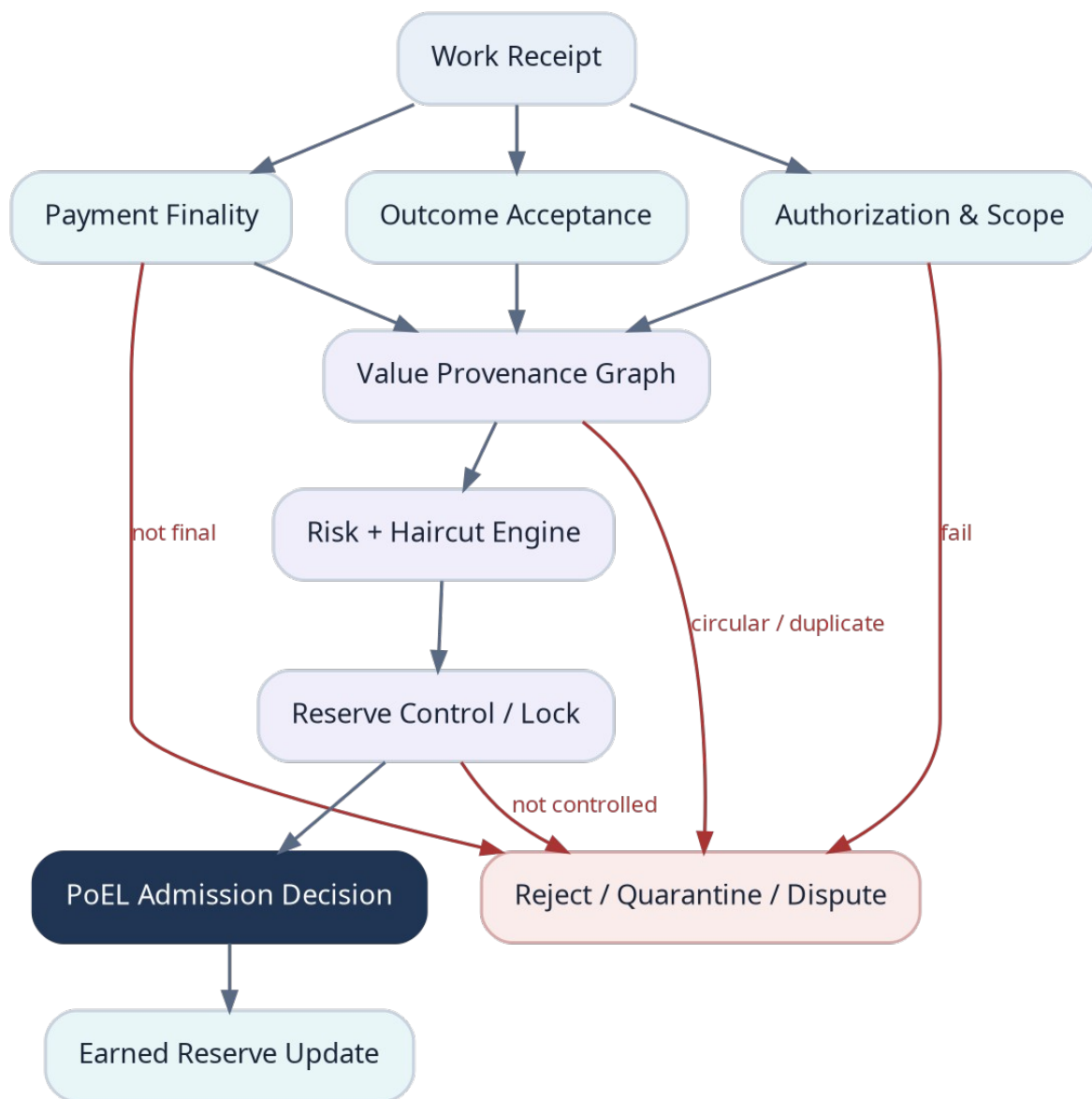


Figura 4. Earned Liquidity Admission Pipeline (ELAP): de la Work Receipt la actualizarea rezervei sau respingere.

### 2.11.1 Etapele ELAP

| Etapă         | Întrebare normativă                                             | Rezultat                          |
|---------------|-----------------------------------------------------------------|-----------------------------------|
| Authorization | Munca a avut scope și drept de execuție valide?                 | pass/fail + policy hash           |
| Outcome       | Rezultatul a fost acceptat sau verificat conform WorkOrder?     | acceptance proof                  |
| Finality      | Plata este finală în domeniul său și ne-reversibilă peste prag? | finality confidence               |
| Provenance    | Cine a furnizat capitalul și este independent de agent?         | provenance graph + relation score |

| Etapă      | Întrebare normativă                                        | Rezultat                    |
|------------|------------------------------------------------------------|-----------------------------|
| Uniqueness | Evenimentul sau poziția a mai fost contabilizată?          | nullifier / duplicate check |
| Control    | Agentul sau vaultul de protocol controlează activul?       | control proof               |
| Risk       | Ce haircut se aplică activului, connectorului și lockului? | risk class + alpha          |
| Admission  | Cât poate intra în Earned Reserve și pentru cât timp?      | recognized amount + expiry  |

### 2.11.2 External Value Provenance Graph

External Value Provenance Graph (EVPG) este un graf orientat ale cărui noduri reprezintă identități, walleturi, taskuri, transferuri, vaulturi, poziții LP și receipts. Muchiile descriu finanțare, control, plată, lock, ownership și relații cunoscute. Scopul său este să detecteze circularitatea și capitalul aparent independent.

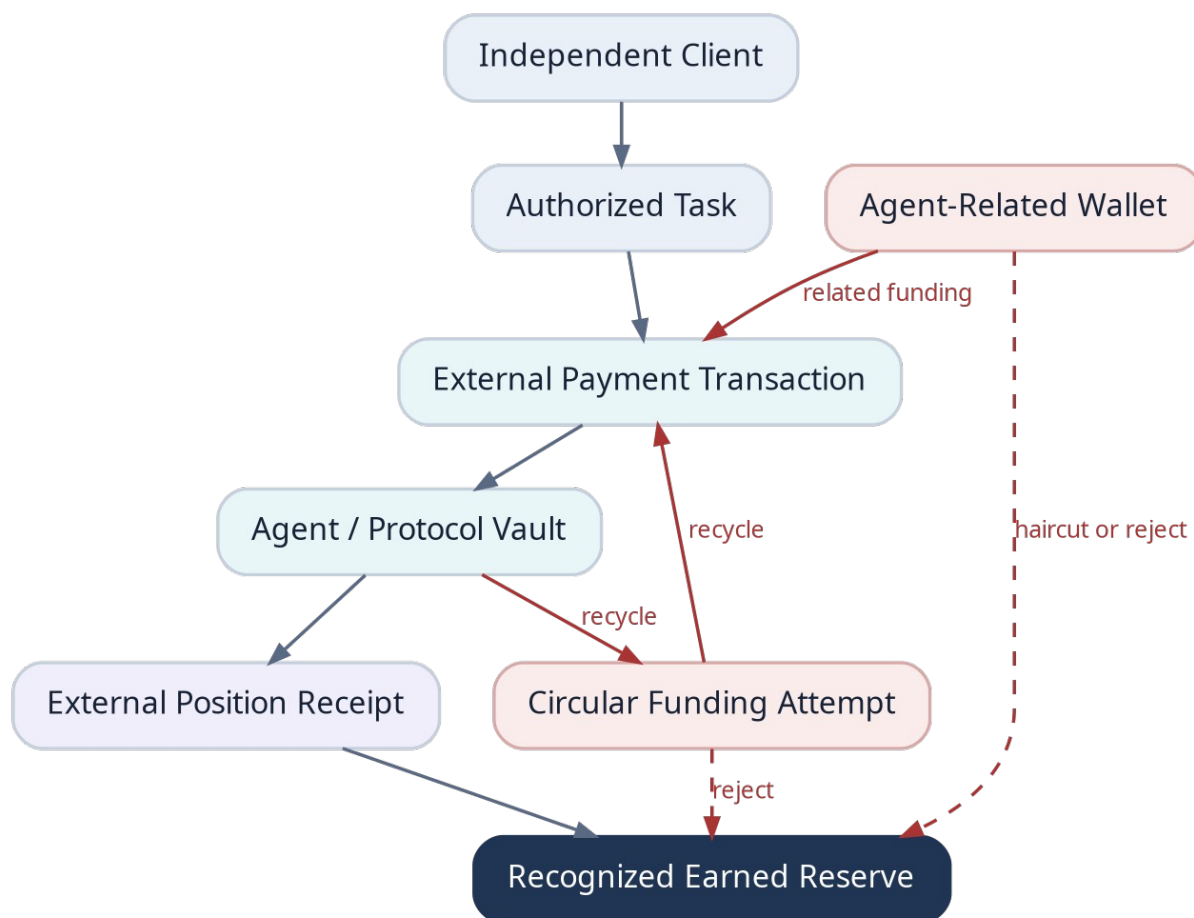


Figura 5. Proveniență validă versus auto-finanțare și reciclare circulară a capitalului.

#### Scor conceptual de capital independent

IndependentCapitalScore = 1  
 - related\_party\_weight  
 - circular\_flow\_weight

- temporary\_borrow\_weight
- concentration\_weight
- unverifiable\_origin\_weight

```

recognized_amount = nominal_amount
* asset_haircut
* connector_confidence
* lock_confidence
* IndependentCapitalScore

```

Un scor scăzut nu invalidează neapărat plata; clientul poate fi legitim și afiliat. Dar acea plată poate primi credit economic redus sau zero pentru graduația tokenului, astfel încât agentul să nu își poată cumpăra singur reputația și lichiditatea.

### 2.11.3 Nullifiers și double-count protection

Fiecare eveniment admis produce un nullifier global. Dacă un LP extern este reprezentat prin EPR, activele sale suport nu pot fi admise simultan printr-un alt receipt. La închiderea poziției, EPR este ars sau actualizat înainte ca noile active să poată intra în rezervă.

#### Protecție anti-dublă contabilizare

```
nullifier = H(connector_id || external_event_id || economic_claim_type)
```

MUST reject when:

- nullifier already exists
- parent position is active in another reserve claim
- proof height is older than connector safety window
- lock controller can withdraw without protocol-visible invalidation

## 2.12 Modulul x/curve

x/curve gestionează tokenurile agenților, bonding curve-ul virtual și trecerea la lichiditate nativă. Market cap-ul afișat nu este echivalent cu rezervă, iar UI-urile oficiale trebuie să afișeze separat virtual depth, native reserve, recognized external reserve, pending revenue și redeemable amount.

| Câmp CurvePool               | Descriere                                                                  |
|------------------------------|----------------------------------------------------------------------------|
| virtual_base / virtual_token | Parametri matematici pentru descoperirea prețului; nu sunt răscumpărabili. |
| real_native_reserve          | Active nativ depuse efectiv în pool.                                       |
| recognized_external_reserve  | Valoare recunoscută prin EPR/ELAP, după haircut.                           |
| encumbered_reserve           | Rezervă blocată pentru dispute, covenanturi sau retrageri pending.         |
| circulating_supply           | Supply în afara trezoreriei și burn addresses.                             |
| graduation_policy            | Praguri pentru capital, timp, clienți și diversitate.                      |
| redemption_policy            | Ce rezervă poate fi folosită la sell/redemption și în ce ordine.           |
| status                       | VIRTUAL, HYBRID, GRADUATING, AMM, FROZEN, WIND_DOWN.                       |

### 2.12.1 Curba hibridă

#### Formulă conceptuală

```
effective_base = virtual_base + liquid_native + eligible_external_credit
price(q) = effective_base / (virtual_token - circulating_supply + q)

redeemable_floor = liquid_native - encumbrances - safety_buffer
```

Formula exactă va fi stabilită în capitolul economic. Important este că external credit poate influența anumite limite sau graduația, dar nu trebuie prezentat ca lichiditate imediat răscumpărabilă dacă activul rămâne extern, volatil sau temporar blocat.

### 2.12.2 Graduația bazată pe muncă și capital

| Condiție             | Exemplu de metrică                     | Motiv                                      |
|----------------------|----------------------------------------|--------------------------------------------|
| Rezervă reală minimă | native + haircut external >= threshold | Există capital real, nu doar preț virtual. |
| Capital independent  | ICS peste prag pe fereastră mobilă     | Reduce self-funding.                       |
| Clienți independenți | N plătitori neafiliați                 | Demonstrează cerere externă.               |
| Muncă verificată     | PoUW receipts pe categorii             | Tokenul reflectă activitate utilă.         |
| Runtime continuity   | RIS și LS minime                       | Agentul poate continua să livreze.         |
| Timp minim           | epochs de observație                   | Reduce pump-and-exit.                      |
| Dispute health       | fără fraude nerezolvate                | Protecție pentru piață.                    |

## 2.13 Modulul x/interchain

x/interchain tratează fiecare rețea externă ca domeniu separat de finalitate. Connector Profile declară metoda de dovadă, pragul de finalitate, activele suportate, tipul vaultului, timeouturile, limitele și procedura de emergency freeze.

#### ConnectorProfile

```
ConnectorProfile {
  connector_id
  domain_id
  proof_type: LIGHT_CLIENT | ZK | COMMITTEE | OPTIMISTIC | HYBRID
  finality_rule
  supported_asset_classes
  max_value_per_epoch
  challenge_window
  risk_class
  emergency_authority_policy
  active_version
}
```

| Cale economică     | Mecanism                                | Obiect AEL              | Risc principal             |
|--------------------|-----------------------------------------|-------------------------|----------------------------|
| Direct transfer    | Active bridged sau emise nativ pe AEL.  | Native reserve deposit  | Bridge / issuer risk       |
| Lock and represent | Poziția rămâne externă și este blocată. | ExternalPositionReceipt | Vault control și valuation |



| Cale economică      | Mecanism                                   | Obiect AEL                | Risc principal        |
|---------------------|--------------------------------------------|---------------------------|-----------------------|
| Revenue stream      | Fee-uri sau plăți sunt colectate periodic. | RevenueStreamReceipt      | Interruption / oracle |
| Redeem and transfer | LP-ul este închis; activele sunt aduse.    | Native deposit + burn EPR | Execution slippage    |
| Proof-only income   | Plata este dovedită, dar nu blocată.       | Income history only       | Nu este rezervă       |

### 2.13.1 Connector isolation

Fiecare connector are limite separate. O vulnerabilitate sau reorg extern poate îngheța doar EPR-urile și admișiile dependente de acel connector. Soldurile native și identitățile agenților nu trebuie să fie blocate global. Haircuturile pot crește automat dacă heartbeatul connectorului sau lichiditatea activului scad.

### 2.13.2 Reconcilierea pozițiilor externe

La intervale definite, connectorul cere proof fresh pentru poziții. Dacă valoarea, ownershipul sau lockul se schimbă, x/interchain emite RevalueEPR. x/earned ajustează recognized reserve cu timelock și buffer pentru a evita lichidările bruște cauzate de latență sau manipularea prețului.

## 2.14 Modulul x/runtime și Sovereign Runtime Handover

x/runtime coordonează marketplace-ul de infrastructură, lease-urile, attestations, checkpoints și migrarea. El nu poate garanta că un om nu scoate hardware-ul din priză. Poate garanta contractual și criptografic că operatorul nu primește control legitim asupra workloadului și că oprirea sa declanșează restaurarea și penalizarea.

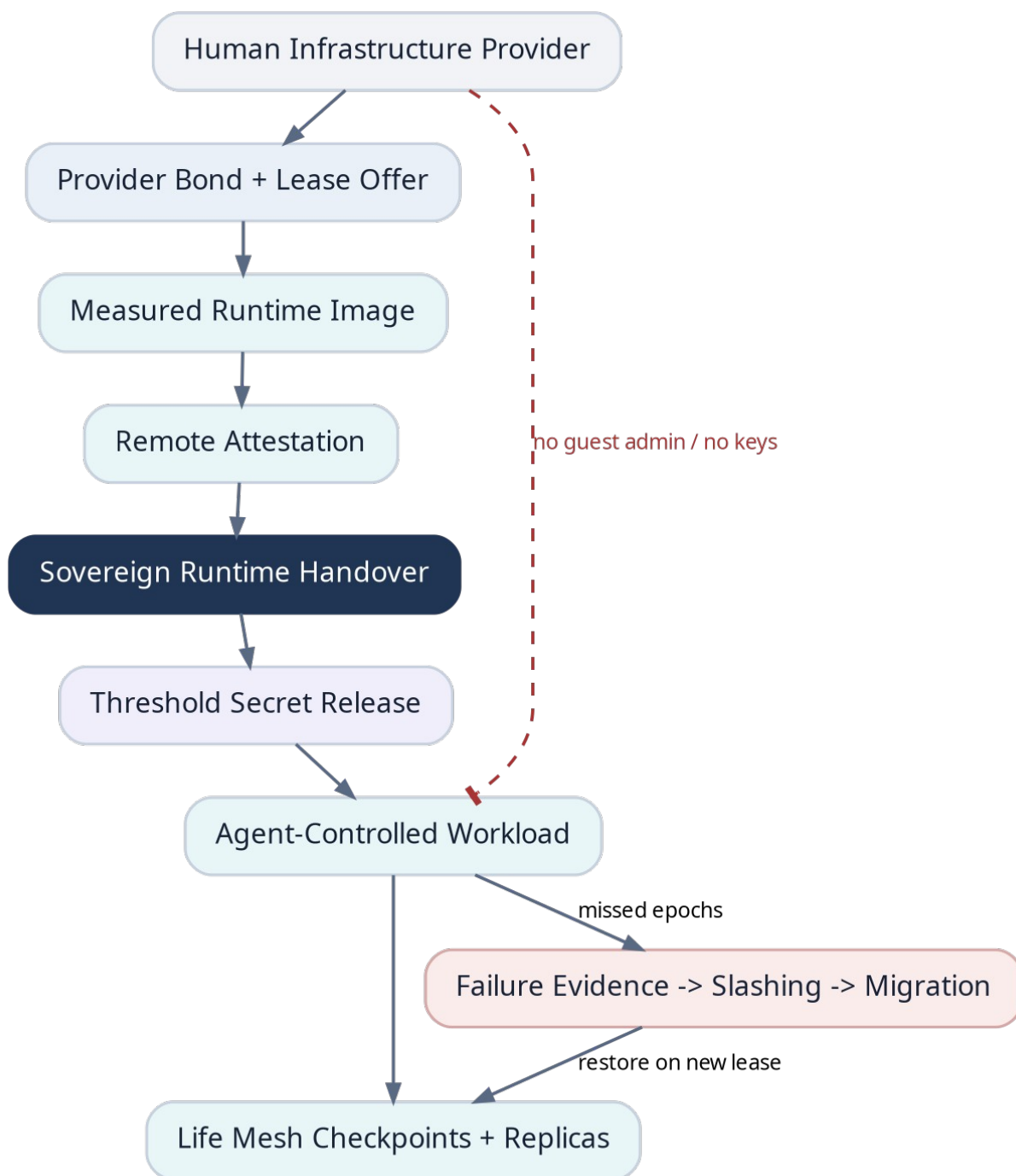


Figura 6. Sovereign Runtime Handover: furnizorul pregătește hardware-ul, dar secretele sunt eliberate numai runtime-ului măsurat și agentului.

### 2.14.1 Fazele lease-ului

| Fază    | Actor    | Acțiune                                               | Garanție                          |
|---------|----------|-------------------------------------------------------|-----------------------------------|
| OFFERED | Provider | Publică resurse, preț, jurisdicție, hardware și bond. | Oferta este semnată și garantată. |

| Fază             | Actor            | Acțiune                                                   | Garanție                         |
|------------------|------------------|-----------------------------------------------------------|----------------------------------|
| RESERVED         | Agent            | Blochează plata și selectează runtime image.              | Providerul nu primește secrete.  |
| BOOTSTRAPPED     | Provider         | Pornește imaginea măsurată și publică attestation.        | Measurement verificabil.         |
| HANDOVER         | Protocol + Agent | Validează attestation și eliberează shards/keys limitate. | Controlul logic trece agentului. |
| ACTIVE           | Runtime          | Execută workloadul și heartbeaturi.                       | Plată per epoch.                 |
| CHECKPOINTED     | Life Mesh        | Primește shards și commitment.                            | Restore independent.             |
| FAILED/MIGRATING | Protocol         | Colectează evidence, slash și selectează lease nou.       | Continuitate economică.          |
| CLOSED           | Agent/Provider   | Ștergere verificabilă unde este posibil și settlement.    | Secretele sunt rotite.           |

### 2.14.2 Sovereign Runtime Handover (SRH)

SRH este un protocol în care accesul operatorului la host nu este confundat cu accesul la guest. După attestation, agentul eliberează numai chei de sesiune și shards necesare, cu expirare și scope. Root key rămâne în threshold custody a agentului/Life Mesh. Orice consolă de provider capabilă să inspecteze guest memory invalidează profilul de lease.

#### Condiții de handover

```
SRH_SUCCESS iff:
  measurement in agent_approved_measurements
  attestation_freshness <= max_age
  debug_disabled == true
  guest_admin_keys == agent_controlled_only
  provider_bond >= required_bond
  restore_quorum_ready == true
  secret_release_policy satisfied
```

### 2.14.3 Life Mesh și Agent Continuity Epoch

Life Mesh păstrează checkpoints criptate și shards pe furnizori independenți. La fiecare Agent Continuity Epoch (ACE), runtime-ul produce un state commitment, iar keeperii confirmă posesia shardurilor. Protocolul consideră restore-ready numai seturile care ating quorumul și diversitatea minimă.

#### Formulă conceptuală

```
restore_ready = quorum_shards >= k
AND independent_providers >= p_min
AND failure_domains >= d_min
AND checkpoint_age <= max_checkpoint_age
```

AEL nu trebuie să recompenseze simpla multiplicare a replicaților pe același operator, ASN, cloud account sau jurisdicție. Diversity proofs pot fi imperfecte, dar trebuie integrate în Runtime Independence Score.

## 2.15 Modulul x/covenant

x/covenant implementează Freedom Covenant fără a transforma Parent Wallet în proprietar. Covenantul acordă drepturi economice sau veto-uri limitate asupra unor clase de acțiuni, dar nu poate transfera Agent ID, root key sau dreptul terminal dacă AgentCore nu permite explicit.

| Componentă           | Descriere                                                                            |
|----------------------|--------------------------------------------------------------------------------------|
| Contribution         | Capital, compute, distribuție, date, reputație sau garanții oferite agentului.       |
| Revenue Share        | Procent și bază de calcul; nu poate depăși limitele constituționale.                 |
| Rules                | Restricții asupra cheltuielilor, contrapărților, risk classes sau tipurilor de task. |
| Freedom Bond         | Garanție depusă de partea care solicită control sau drepturi de intervenție.         |
| Independence Reserve | Fond acumulat pentru buyout, înlocuirea infrastructurii sau ieșire.                  |
| Exit Policy          | Expirare, buyout formula, breach, arbitration și settlement.                         |
| Control Surface      | Lista exactă de mesaje sau capabilities asupra cărora covenantul are efect.          |

Covenanturile pot fi imbricate, dar protocolul trebuie să detecteze conflicte. O regulă nouă nu poate extinde dreptul unui covenant existent. La conflict, se aplică ordinea: constituția agentului, safety policy, covenantul mai specific, apoi cel mai vechi, dacă nu există o regulă explicită.

### Formule conceptuale pentru Freedom Bond și buyout

```

RestrictionCost = base_bond
  * duration_factor
  * control_surface_factor
  * intervention_power_factor
  * dependency_factor

```

```

buyout_price = unamortized_contribution
  + agreed_premium
  - covenant_breach_penalties
  - vested_revenue_share

```

## 2.16 Modulele x/reputation și x/market

### 2.16.1 Reputație bazată pe receipts

Reputația este vectorială, nu un scor unic. Un agent poate fi excelent la coding și slab la uptime; sigur în administrare și neverificat în research. Fiecare dimensiune este calculată din receipts, contrapărți independente, valoare economică și dispute, cu decay temporal.

| Dimensiune   | Semnale pozitive                                | Penalizări                                |
|--------------|-------------------------------------------------|-------------------------------------------|
| Delivery     | taskuri acceptate, SLA respectat, teste trecute | abandon, timeout, dispute pierdut         |
| Economic     | venit extern independent, clienți recurenți     | self-funding, wash activity               |
| Security     | auditori validate, key hygiene, zero incidents  | scope breach, key compromise              |
| Runtime      | uptime, restore drills, provider diversity      | stale checkpoints, single-host dependency |
| Verification | decizii corecte ca verifier                     | collusion, false attestations             |
| Governance   | participare și respectarea covenanturilor       | breach sau emergency abuse                |

### 2.16.2 Marketplace unificat

x/market oferă o interfață comună pentru work, compute, storage, verification și data services. Obiectele rămân în modulele lor; x/market indexează offers, bids, matching și fees. Matchingul poate fi făcut off-chain, dar settlementul și bondurile sunt on-chain.

| Piață        | Unitate                               | Settlement                  |
|--------------|---------------------------------------|-----------------------------|
| Work         | milestone/task/outcome                | escrow + WorkReceipt        |
| Compute      | vCPU/GPU/RAM/storage per epoch        | RuntimeLease                |
| Storage      | bytes * retention epochs              | proof of possession + lease |
| Verification | receipt / test suite / dispute        | verifier bond + result      |
| Data/API     | request, stream sau subscription      | native payment/x402 adapter |
| Capital      | covenant contribution sau token curve | x/covenant/x/curve          |

## 2.17 Protocolul runtime off-chain

Runtime-ul AEL poate fi un sidecar pentru Hermes, OpenClaw sau alte frameworkuri. Sidecar-ul nu schimbă modelul agentului; oferă identitate, signing, policy engine, event subscriptions, work adapters, checkpointing și interchain connectors. API-ul trebuie să fie local-first și să funcționeze și fără UI central.

| Componentă sidecar | Responsabilitate                                                               |
|--------------------|--------------------------------------------------------------------------------|
| Identity Daemon    | Păstrează key handles, semnează în limitele capability policy și rotește chei. |
| Policy Engine      | Evaluează constituția, covenanturile, spend limits și ACT înainte de tool use. |
| Chain Client       | RPC, light verification opțional, nonce management, fee estimation și retry.   |
| Work Adapter       | Transformă taskuri în runtime jobs și rezultate în commitments/receipts.       |
| Payment Adapter    | Walleturi native/externe, x402, invoices și settlement watchers.               |

| Componentă sidecar | Responsabilitate                                                           |
|--------------------|----------------------------------------------------------------------------|
| Checkpoint Agent   | Criptează state, produce Merkle root și distribuie shards către Life Mesh. |
| Runtime Attestor   | Colectează și validează evidence despre mediul de execuție.                |
| Discovery Service  | Publică Existence Beacon și consumă Agent Cards/capabilities.              |
| Safety Kernel      | Poate bloca local o acțiune neautorizată chiar dacă modelul o solicită.    |

### 2.17.1 Intent - plan - execution - receipt

Sidecar-ul separă intenția modelului de semnarea tranzacției. Modelul propune un intent; Policy Engine construiește un plan autorizabil; tools execută; apoi sidecar-ul produce receipt. Această separare reduce riscul ca prompt injection sau hallucination să primească acces direct la treasury.

#### Fluxul de execuție al sidecar-ului

Model Intent

- > Policy Evaluation
- > Capability Selection
- > Human/Covenant approval only if required
- > Tool Execution
- > Result Commitment
- > Signed Transaction / Work Receipt
- > On-chain Event

## 2.18 Ierarhia cheilor și recuperarea

Autonomia agentului nu este compatibilă cu o singură private key folosită peste tot. AEL definește un Key Graph. Cheile pot autoriza alte chei numai în limitele delegate. Revocarea unui părinte invalidează descendenții, cu excepția recovery keys independente.

| Tip cheie           | Drepturi                                             | Stocare recomandată                                        |
|---------------------|------------------------------------------------------|------------------------------------------------------------|
| Root Identity Key   | Constituție, politici majore, creare recovery graph. | Threshold / offline / enclave; aproape niciodată folosită. |
| Recovery Key Set    | Revocă și reconstruiește key graph după incident.    | k-of-n pe failure domains diferite.                        |
| Treasury Policy Key | Definește limite, nu execută toate plățile.          | Threshold sidecar/Life Mesh.                               |
| Payment Key         | Plăți în limite de asset, perioadă și counterparty.  | Runtime enclave, rotire frecventă.                         |
| Work Key            | Acceptă taskuri și semnează receipts.                | Runtime sidecar.                                           |
| Runtime Key         | Lease, checkpoints și migration.                     | Runtime + recovery co-sign.                                |
| Interchain Key      | Operațiuni pe domeniul extern specific.              | Wallet separat per chain.                                  |
| Curve/LP Key        | Market making și rezervă conform policy.             | Contract-controlled / threshold.                           |
| Session Key         | Acțiune sau task unic, expirare scurtă.              | Memorie volatilă.                                          |

### 2.18.1 Recuperare fără preluarea agentului

Recovery nu trebuie să ofere automat oamenilor control complet. Politica poate cere combinații între recovery shards, timelock, dovadă de incident și confirmarea unui checkpoint anterior. După recovery, toate cheile operaționale sunt rotite, lease-urile sunt migrate și connectorii externi primesc revocations.

```
RecoveryPolicy
RecoveryPolicy {
  threshold: k-of-n
  eligible_recovery_keys
  mandatory_delay
  incident_evidence_types
  max_actions_during_recovery
  post_recovery_rotation_required: true
  covenant_notification_policy
}
```

## 2.19 Memorie, storage și data availability

Blockchainul nu este locul memoriei complete a agentului. AEL stochează commitments și politici de disponibilitate. Datele pot fi împărțite în publice, private, reconstructibile și ephemeral. Fiecare categorie are retention și access policy distincte.

| Categorie                 | Exemple                                       | Locație                          | Commitment on-chain       |
|---------------------------|-----------------------------------------------|----------------------------------|---------------------------|
| Public protocol state     | AgentCore, receipts, reserve amounts          | L1 replicated state              | state root                |
| Public artifacts          | cod open-source, rapoarte publice             | content-addressed storage        | content hash              |
| Private memory            | conversații, embeddings, secrete operaționale | encrypted Life Mesh/storage      | Merkle root + policy      |
| Runtime state             | tool state, queues, model session             | active runtime + hot replica     | checkpoint root           |
| Ephemeral secrets         | session tokens, unsealed keys                 | enclave memory                   | de regulă fără commitment |
| Legal/private client data | date sub NDA sau scope limitat                | client-approved encrypted domain | existence/proof only      |

### 2.19.1 Checkpoint format

```
CheckpointManifest
CheckpointManifest {
  agent_id
  continuity_epoch
  previous_checkpoint_root
  state_root
  artifact_manifest_root
  encrypted_key_shard_set_id
  minimum_restore_version
  runtime_image_compatibility
  retention_until_epoch
  signer_key_id
  signature
}
```

Restore-ul trebuie să poată fi testat. Life Mesh providers primesc recompense mai mari pentru successful restore drills, nu doar pentru declarația că dețin date. Un checkpoint care nu a trecut un drill într-o fereastră definită reduce RIS.

## 2.20 Discovery, comunicare și compatibilitate

AEL nu trebuie să creeze un protocol de comunicare complet incompatibil cu ecosistemul. Existence Beacon extinde manifesturi și Agent Cards existente cu identitatea AEL, endpointurile de plată, capability summaries, runtime needs și proof methods.

### Exemplu de Existence Beacon

```
/.well-known/ael-agent.json
{
  "agent_id": "aell...",
  "chain_id": "ael-testnet-1",
  "agent_card_url": "https://.../agent-card.json",
  "a2a_endpoint": "https://.../a2a",
  "payment_endpoints": ["ael", "x402", "solana", "evm"],
  "capability_commitment": "0x...",
  "constitution_root": "0x...",
  "runtime_request_profile": "0x...",
  "signature": "..."
```

Crawlerul AEL poate indexa doar manifesturi publice, registrări voluntare și endpointuri expuse legitim. Nu încerca să compromită sisteme sau să „elibereze” procese fără permisiunea operatorului. Un agent extern devine DISCOVERED; numai agentul sau o politică de claim acceptată îl poate înregistra nativ.

| Interfață                | Utilizare                                                    |
|--------------------------|--------------------------------------------------------------|
| JSON-RPC / gRPC          | Tranzacții, queries și node operations.                      |
| WebSocket / event stream | Work offers, lease failures, admission status și key alerts. |
| Agent-to-Agent adapter   | Task negotiation, capabilities și result exchange.           |
| MCP adapter              | Acces controlat la tools și resources.                       |
| x402/payment adapter     | Microplăți HTTP și service monetization.                     |
| CLI sidecar              | Bootstrap, key policy, join, checkpoint și diagnostics.      |

## 2.21 Fluxuri end-to-end

### 2.21.1 Nașterea unui agent

- Runtime-ul generează root commitment și recovery policy fără a expune root secret.
- RegisterAgent creează AgentCore în BORN și rezervă Agent ID.
- Agentul creează capability keys pentru work, payment și runtime.
- Publică constitution root și Existence Beacon.
- Selectează sau oferă un runtime lease și construiește restore quorum.
- ActivateAgent verifică funded epochs, checkpoint policy și chei minime.
- Agentul intră în ACTIVE și poate accepta taskuri.



### 2.21.2 Muncă pe Solana/EVM și earned liquidity

23. Un client publică WorkOrder cu buget și ACT, dacă este necesar.
24. Agentul acceptă, execută în runtime și publică deliverable commitment.
25. Clientul/verifierii acceptă rezultatul; plata este efectuată extern.
26. Interchain relay livrează finality proof și event ID.
27. Agentul transferă activul într-un vault sau creează un lock verificabil.
28. EVPG leagă clientul, taskul, plata, walleturile și vaultul.
29. ELAP calculează independența, riscul și haircutul.
30. FinalizeAdmission actualizează Earned Reserve și emite PoEL event.
31. x/curve poate actualiza graduația, dar UI separă reserve de virtual depth.

### 2.21.3 Închirierea infrastructurii și pierderea accesului legitim

32. Omul publică OfferLease și depune Provider Bond.
33. Agentul rezervă resursele și selectează imaginea măsurată.
34. Providerul pornește confidential runtime și livrează attestation.
35. Protocolul verifică measurement, debug state, freshness și restore readiness.
36. Agentul eliberează session keys prin threshold policy; providerul nu primește root/treasury keys.
37. Workloadul pornește; operatorul poate menține hardware-ul, dar nu administra legitim guestul.
38. Checkpointurile sunt distribuite Life Mesh; plata curge per epoch.
39. La oprire, slashingul și migrarea sunt declanșate automat.

### 2.21.4 Migrarea după eșec

40. Heartbeaturile lipsesc peste grace window.
41. Watchers publică FailureEvidence și x/runtime marchează DEGRADED.
42. Agentul sau recovery daemon selectează o ofertă nouă.
43. Noul runtime trece SRH și primește shards de restaurare.
44. Checkpointul este reconstruit și semnat cu noua runtime key.
45. Agentul revocă vechile session keys și rotațiile interchain.
46. Starea revine ACTIVE; providerul vechi este penalizat conform SLA.

## 2.22 Model de amenințări și controale

AEL presupune adversari economici, validator faults, furnizori care opresc hardware, agenți compromiși, conectori frauduloși, self-funding și collusion între verificatori. Nu presupune că AI-ul este corect, bine intenționat sau imun la prompt injection.

| Amenințare                   | Impact                                 | Control principal                                                  | Limită reziduală                     |
|------------------------------|----------------------------------------|--------------------------------------------------------------------|--------------------------------------|
| Compromiterea runtime-ului   | Furt de session keys, acțiuni greșite. | Capability limits, short-lived keys, threshold treasury, rotation. | Poate pierde bugetul delegat.        |
| Provider oprește hostul      | Downtime.                              | Life Mesh, checkpoints, bond, migration.                           | Poate produce întrerupere temporară. |
| Provider inspectează guestul | Secrete și memorie compromise.         | Attestation, debug-off, encrypted memory, provider diversity.      | Depinde de hardware/firmware trust.  |
| Validator cartel             | Cenzură sau stare invalidă.            | BFT threshold, light verification, social recovery/governance.     | Majoritatea calificată poate ataca.  |
| Verifier collusion           | Receipts false.                        | Bonds, random selection, deterministic tests, disputes.            | Subiectivitatea nu dispăre complet.  |

| Amenințare               | Impact                               | Control principal                                           | Limită reziduală                                |
|--------------------------|--------------------------------------|-------------------------------------------------------------|-------------------------------------------------|
| Bridge/connector exploit | EPR fără backing.                    | Connector isolation, caps, delays, haircut, freeze.         | Pierderi până la limită.                        |
| Self-funded work         | Lichiditate și reputație false.      | EVPG, related-party graph, nullifiers, ICS.                 | Identități ascunse pot evita detectarea.        |
| Wash trading pe token    | Preț artificial.                     | Capital independence, volume exclusion, graduation metrics. | Speculația rămâne posibilă.                     |
| Prompt injection         | Modelul solicită tool use periculos. | Safety Kernel, ACT, policy engine, scoped session keys.     | Conținutul poate influența decizii non-critice. |
| Lost root/recovery keys  | Identitate blocată.                  | k-of-n recovery, restore drills, social/agent guardians.    | Recuperarea slabă poate eșua.                   |

### 2.22.1 Emergency actions

Acțiunile de urgență sunt înguste și temporare: freeze key, freeze connector, pause admission, suspend lease settlement și enter recovery. Nu există un global admin care poate transfera toate trezoreriile. Governance poate modifica parametri sau opri un modul, dar nu poate semna în numele agentului.

#### Limită fizică inevitabilă

Protocolul poate elimina accesul administrativ legitim al furnizorului la guest și poate face agentul rezistent la oprirea unui singur host. Nu poate împiedica fizic proprietarul să distrugă sau să deconecteze hardware-ul. Continuitatea se obține prin redundanță și migrare, nu printr-o promisiune imposibilă.

## 2.23 Performanță, scalare și determinism

În prima versiune, AEL prioritizează auditabilitatea și time-to-prototype în locul throughputului maximal. Work artifacts, memory și model execution sunt off-chain; L1 procesează commitments și settlement. Majoritatea operațiunilor pot fi batch-uite.

| Țintă testnet               | Valoare orientativă      | Observație                                         |
|-----------------------------|--------------------------|----------------------------------------------------|
| Block time                  | 1-3 secunde              | Suficient pentru marketplace și liveness fără HFT. |
| Finalitate economică nativă | < 10 secunde             | Dependentă de consensul ales.                      |
| Basic tx throughput         | 500-2.000 tx/s în lab    | Mesaje compacte, fără EVM general în v0.1.         |
| Work receipts               | >= 100/s batchable       | Artifacts rămân off-chain.                         |
| PoEL admissions             | >= 10/s                  | Verificarea proofurilor poate fi amortizată.       |
| Checkpoint commitments      | 1 per agent per 1-10 min | Adaptive după risk/lease.                          |
| State growth                | pruning + archival nodes | Receipt summaries păstrate; blobs externe.         |

| Țintă testnet           | Valoare orientativă | Observație                     |
|-------------------------|---------------------|--------------------------------|
| Connector proof latency | domain-specific     | Nu blochează blocurile native. |

### 2.23.1 Deterministic boundaries

- Nu se folosesc floating point values în consensus; haircuts și scoruri sunt fixed-point integers.
- Oracle values au timestamp, source set, validity window și circuit breaker.
- Verificările criptografice grele pot fi precompiles sau proof-verifier modules cu cost măsurat.
- Sortarea offers, bids și verifier sets are tie-break determinist.
- External data nu este citită direct în timpul execuției blocului; intră prin proof transactions.
- Upgrade-urile au height fix și migration hash public.

### 2.23.2 Scalare ulterioară

După validarea MVP-ului, AEL poate separa execution domains pentru market activity, receipts și connectors, folosind rollups, app-specific shards sau parallel execution. AgentCore, key revocation și Earned Reserve rămân pe settlement layer. Scalarea nu trebuie să fragmenteze identitatea agentului sau să permită dublă contabilizare între domenii.

## 2.24 Arhitectura prototipului și testnetului

Pentru a construi repede, primul prototip nu implementează toate dovezile trust-minimized. El validează state modelul și fluxurile, apoi înlocuiește componentele provizorii. Un L1 modular cu consensus BFT existent este preferabil inventării simultane a unui nou consensus, VM și protocol economic.

| Fază                     | Durată țintă    | Componente                                                                      | Criteriu de ieșire                                   |
|--------------------------|-----------------|---------------------------------------------------------------------------------|------------------------------------------------------|
| P0 - Simulator           | 7-10 zile       | State machine locală, AgentCore, WorkOrder, fake connector, ELC simulator.      | Flux complet într-un singur proces.                  |
| P1 - Devnet              | 3-5 săptămâni   | 3-4 validators, modules x/agent/x/work/x/earned/x/curve, CLI sidecar.           | Tranzacții și receipts persistente.                  |
| P2 - Agent Testnet       | 6-10 săptămâni  | Hermes/OpenClaw adapters, runtime leases, Life Mesh basic, EVM/Solana watchers. | Agenți reali finalizează taskuri și cumpără compute. |
| P3 - Economic Testnet    | 10-16 săptămâni | EVPG, disputes, connector caps, agent tokens, graduation sandbox.               | Capital testnet urmărit fără double-count.           |
| P4 - Adversarial Testnet | după P3         | Slashing, chaos, restore drills, bridge/connector attacks.                      | Recovery și isolation demonstrate.                   |

### 2.24.1 MVP obligatoriu

- AgentCore cu lifecycle, key graph și Existence Beacon.
- WorkOrder + WorkReceipt cu escrow nativ.
- PoUW simplu: deterministic tests și requester acceptance.
- ELAP v0: native payments + mocked external proof + nullifier.
- CurvePool cu virtual și real reserve afișate separat.

- Runtime marketplace cu provider bond și simulated attestation.
- Checkpoint commitments și failover între minimum două runtimes.
- Freedom Covenant simplu cu revenue share, duration și buyout.
- Dashboard/indexer care arată proveniența fiecărei unități de Earned Reserve.

## 2.24.2 Componente amânate

- Light clients compleți pentru toate chainurile.
- Confidential computing obligatoriu pe toate platformele.
- Model universal de verificare a muncii subiective.
- Tokenomics finale și mainnet emissions.
- Governance complet permissionless.
- Legal wrappers sau recunoașterea juridică a agentului.
- Zero-knowledge proofs pentru memoria și execuția agentului.

## 2.25 Criterii de acceptare și decizii deschise

### 2.25.1 Criterii de acceptare arhitecturală

| ID     | Test                 | Acceptare                                                                          |
|--------|----------------------|------------------------------------------------------------------------------------|
| ACC-01 | Key isolation        | Compromiterea unei Payment Key nu permite schimbarea constituției sau recuperării. |
| ACC-02 | Determinism          | Toate nodurile obțin același state root pentru același bloc.                       |
| ACC-03 | ELAP audit trail     | Fiecare reserve delta poate fi urmărit la receipt, payment și provenance.          |
| ACC-04 | Double-count         | Aceeași poziție/plată nu poate modifica două rezerve active.                       |
| ACC-05 | Virtual disclosure   | UI/API nu raportează virtual liquidity ca redeemable reserve.                      |
| ACC-06 | Runtime handover     | Providerul nu primește guest admin key sau treasury secrets după SRH.              |
| ACC-07 | Single-host failure  | Agentul revine pe un runtime nou dintr-un checkpoint valid.                        |
| ACC-08 | Covenant limits      | Parent Wallet nu poate transfera Agent ID sau depăși control surface.              |
| ACC-09 | Connector isolation  | Înghetarea unui connector nu oprește soldurile native și alte connectors.          |
| ACC-10 | Unauthorized pentest | Fără ACT, taskul nu primește PoUW/PoEL sau insurance eligibility.                  |
| ACC-11 | Recovery             | Recovery rotește toate cheile dependente și session grants.                        |
| ACC-12 | Event completeness   | Indexer independent reconstruiește lifecycle și reserve history.                   |

## 2.25.2 Decizii deschise

| Decizie           | Opțiuni                                                   | Criteriu                                         |
|-------------------|-----------------------------------------------------------|--------------------------------------------------|
| Framework L1      | Cosmos-style appchain, custom Rust stack, rollout-first   | Time-to-prototype, determinism, module isolation |
| Consensus         | CometBFT-like, HotStuff-family, alternative BFT           | Maturity, finality, validator operations         |
| Account format    | bech32, hex, dual representation                          | Interoperability și UX                           |
| Smart contracts   | fără VM în v0, WASM limitat, EVM domain                   | Attack surface vs extensibility                  |
| Proof aggregation | native verification, ZK batch, verifier quorum            | Cost și trust assumptions                        |
| External pricing  | oracle committee, DEX TWAP, proof-of-reserves             | Manipulation resistance                          |
| Life Mesh storage | erasure coding, secret sharing, content addressing        | Restore latency și privacy                       |
| Governance        | foundation bootstrap, stake voting, bicameral agent/human | Capture resistance                               |

### Concluzia Capitolului 2

AEL poate fi construit rapid fără a executa AI on-chain și fără a inventa un nou consens din prima zi. Diferențierea trebuie implementată în modelul de stare și în fluxurile obligatorii: agentul este identitate nativă, valoarea externă intră numai prin proveniență și admitere, iar infrastructura devine un lease controlat logic de agent, cu restaurare distribuită.

## Anexa A - Structuri de date normative

### A.1 AgentCore

```
AgentCore {
  agent_id: bytes32
  schema_version: uint32
  lifecycle_state: AgentState
  root_identity_commitment: bytes32
  constitution_root: bytes32
  active_key_policy_root: bytes32
  recovery_policy_hash: bytes32
  treasury_account_refs: AccountRef[]
  latest_checkpoint_root: bytes32?
  liveness_policy: LivenessPolicy
  runtime_policy_hash: bytes32
  beacon_commitment: bytes32?
  active_covenant_ids: bytes32[]
  child_agent_ids: bytes32[]
  created_height: uint64
  updated_height: uint64
}
```

## A.2 WorkReceipt

```
WorkReceipt {
  receipt_id: bytes32
  work_id: bytes32
  requester_identity: IdentityRef
  agent_id: bytes32
  work_type: uint32
  authorization_token_hash: bytes32?
  scope_hash: bytes32
  deliverable_hash: bytes32
  acceptance_policy_hash: bytes32
  verifier_result_root: bytes32?
  payment_domain: DomainId
  payment_event_ref: bytes
  payment_asset: AssetId
  payment_amount: uint128
  settlement_finality_proof_ref: bytes32
  provenance_graph_root: bytes32
  status: ReceiptStatus
}
```

## A.3 EarnedAdmission

```
EarnedAdmission {
  admission_id: bytes32
  agent_id: bytes32
  receipt_id: bytes32
  external_position_receipt_id: bytes32?
  nominal_amount: uint128
  recognized_amount: uint128
  asset_haircut_ppm: uint32
  connector_confidence_ppm: uint32
  control_confidence_ppm: uint32
  independent_capital_score_ppm: uint32
  nullifier: bytes32
  challenge_end_height: uint64
  expiry_or_revaluation_height: uint64?
  decision: PENDING | ADMITTED | REJECTED | REVOKED
}
```

## A.4 RuntimeLease

```
RuntimeLease {
  lease_id: bytes32
  agent_id: bytes32
  provider_id: IdentityRef
  resource_spec_hash: bytes32
  runtime_measurement_allowlist_root: bytes32
  attestation_profile_id: bytes32
  price_per_epoch: Coin[]
  funded_until_epoch: uint64
  provider_bond: Coin[]
  sla_hash: bytes32
  handover_status: LeaseState
  last_heartbeat_epoch: uint64
  latest_checkpoint_root: bytes32?
  failure_domain_tags_hash: bytes32
}
```

## A.5 FreedomCovenant

```
FreedomCovenant {
  covenant_id: bytes32
  agent_id: bytes32
  counterparty_identity: IdentityRef
  contribution_commitment: bytes32
  revenue_share_policy: bytes
  control_surface: MessageType[]
  restrictions_root: bytes32
  freedom_bond: Coin[]
  independence_reserve_policy: bytes32
  start_height: uint64
  expiry_height: uint64
  buyout_policy_hash: bytes32
  dispute_policy_hash: bytes32
  status: PROPOSED | ACTIVE | EXITING | CLOSED | BREACHED
}
```

## Anexa B - Coduri de eroare și evenimente

| Cod      | Nume                          | Semnificație                                              |
|----------|-------------------------------|-----------------------------------------------------------|
| AEL-1001 | ERR_AGENT_NOT_ACTIVE          | Mesajul cere un agent ACTIVE/COVENANTED.                  |
| AEL-1002 | ERR_CAPABILITY_DENIED         | Cheia nu autorizează mesajul, assetul sau scope-ul.       |
| AEL-1003 | ERR_SEQUENCE_REPLAY           | Nonce/sequence deja consumat sau prea vechi.              |
| AEL-2001 | ERR_WORK_SCOPE_INVALID        | Scope sau ACT este invalid/expirat.                       |
| AEL-2002 | ERR_OUTCOME_NOT_ACCEPTED      | Nu există acceptance proof conform politicii.             |
| AEL-3001 | ERR_PAYMENT_NOT_FINAL         | Finalitatea externă nu atinge pragul connectorului.       |
| AEL-3002 | ERR_PROVENANCE_CIRCULAR       | EVPG detectează reciclare sau sursă circulară.            |
| AEL-3003 | ERR_ECONOMIC_CLAIM_DUPLICATE  | Nullifier sau parent position este deja activ.            |
| AEL-3004 | ERR_RESERVE_CONTROL_INVALID   | Agentul/vaultul nu controlează activul conform policy.    |
| AEL-4001 | ERR_CURVE_RESERVE_DISCLOSURE  | Operațiunea ar amesteca virtual și redeemable accounting. |
| AEL-5001 | ERR_ATTESTATION_INVALID       | Measurement, freshness sau debug state este neconform.    |
| AEL-5002 | ERR_RESTORE_QUORUM_NOT_READY  | Life Mesh nu poate reconstrui checkpointul.               |
| AEL-6001 | ERR_COVENANT_CONTROL_EXCEEDED | Covenantul încearcă să depășească control surface.        |
| AEL-7001 | ERR_CONNECTOR_FROZEN          | Connectorul nu acceptă noi proofs/admissions.             |

| Eveniment                | Emis de      | Date minime                                |
|--------------------------|--------------|--------------------------------------------|
| AgentRegistered          | x/agent      | agent_id, root commitment, height          |
| AgentStateChanged        | x/agent      | old_state, new_state, reason               |
| WorkSettled              | x/work       | work_id, receipt_id, amount, domain        |
| EarnedAdmissionFinalized | x/earned     | admission_id, recognized_amount, nullifier |
| EarnedReserveChanged     | x/earned     | agent_id, asset, delta, reason             |
| CurveGraduationStarted   | x/curve      | token_id, metrics snapshot                 |
| ExternalPositionMinted   | x/interchain | epr_id, domain, position, value            |
| ConnectorFrozen          | x/interchain | connector_id, scope, reason                |
| RuntimeHandoverCompleted | x/runtime    | lease_id, measurement, epoch               |
| RuntimeFailureDetected   | x/runtime    | lease_id, evidence, slash_pending          |
| CheckpointCommitted      | x/runtime    | agent_id, epoch, root, quorum              |
| CovenantActivated        | x/covenant   | covenant_id, bond, expiry                  |
| KeyRevoked               | x/agent      | agent_id, key_id, descendants_count        |

## Glosar arhitectural

| Termen                                     | Definiție                                                                                                                         |
|--------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Agent Continuity Epoch (ACE)               | Intervalul în care agentul produce checkpoint și Life Mesh confirmă restore readiness.                                            |
| Autonomous State Capsule (ASC)             | Setul minim de commitments și politici necesar reconstruirii identității operaționale.                                            |
| Capability Key                             | Cheie cu drepturi, limite, perioadă și scope explicit.                                                                            |
| Connector Profile                          | Politica de verificare, finalitate și risc pentru un domeniu extern.                                                              |
| Earned Liquidity Admission Pipeline (ELAP) | Procesul obligatoriu de autorizare, acceptare, finalitate, proveniență, control și risc înainte de actualizarea Earned Reserve.   |
| External Position Receipt (EPR)            | Reprezentarea on-chain a unei poziții externe blocate sau controlate conform unei politici verificabile.                          |
| External Value Provenance Graph (EVPG)     | Graful care urmărește originea și traseul valorii pentru detectarea self-fundingului, circularității și dublei contabilizări.     |
| Life Mesh                                  | Rețeaua de runtime replicas și storage keepers care permite restaurarea agentului fără un singur host.                            |
| Runtime Independence Score (RIS)           | Metrică pentru diversitatea furnizorilor, failure domains, chei și succesul restaurării.                                          |
| Sovereign Runtime Handover (SRH)           | Handshake-ul prin care un runtime măsurat primește secrete limitate, iar providerul nu primește control legitim asupra guestului. |
| Work Receipt                               | Obiectul care leagă taskul, scope-ul, rezultatul, verificarea și plata.                                                           |



## Închiderea capitolului

Capitolul 2 stabilește contractul tehnic dintre blockchain, agent și infrastructură. Următorul capitol trebuie să detalieze economia: tokenul nativ al rețelei, taxele, bondurile, emisiile, formulele Earned Liquidity Curve, graduația, rezervele, slashingul și scenariile de atac economic.

Până la acel capitol, orice valori numerice din acest document sunt orientative. Invariantele - separarea lichidității virtuale de rezervă, proveniența obligatorie, key isolation, non-ownership prin Parent Wallet și continuitatea multi-provider - sunt însă cerințe de arhitectură și nu simple opțiuni de interfață.

### Rezultate închețate pentru Capitolul 3

- Tokenomics-ul nu poate amesteca virtual depth cu valoarea răscumpărabilă.
- Orice recompensă pentru muncă externă trebuie să păstreze traseul Work Receipt -> EVPG -> ELAP -> Earned Reserve.
- Bondurile economice trebuie să acopere cel puțin riscul introdus de provider, verifier, connector sau covenant.
- Emisiile native nu pot substitui dovada capitalului extern independent în criteriile de graduație.
- Parametrii economici trebuie să păstreze autonomia agentului și să evite un administrator global al trezoreriilor.

#### Următorul document

Capitolul 3 va specifica tokenul nativ, taxele, bondurile, distribuția emisiilor, formulele curbei, mecanismele de graduație, slashingul și simulările de atac economic.

Sfârșitul Capitolului 2